3_Ejercicios_SQLite_Resuelto

September 23, 2026

Creado por:

Isabel Maniega

1 Ejercicio

Tenemos que crear dos tablas dentro de una base de datos llamada "actividad": * edición donde tenemos 2 columnas: * un ID único y que sea incrementado cada vez 1, 2, 3, etc * número donde vendrá en letra el número de edicción: UNO, DOS, etc * notas donde tendremos 5 columnas: * ID único y que sea incrementado cada vez 1, 2, 3, etc * name: nombre de cada alumno (varchar) * edad: edad de los alumnos (int) * notas: notas de los alumnos (real) * ID edic: corresponde al id de la tabla edición.

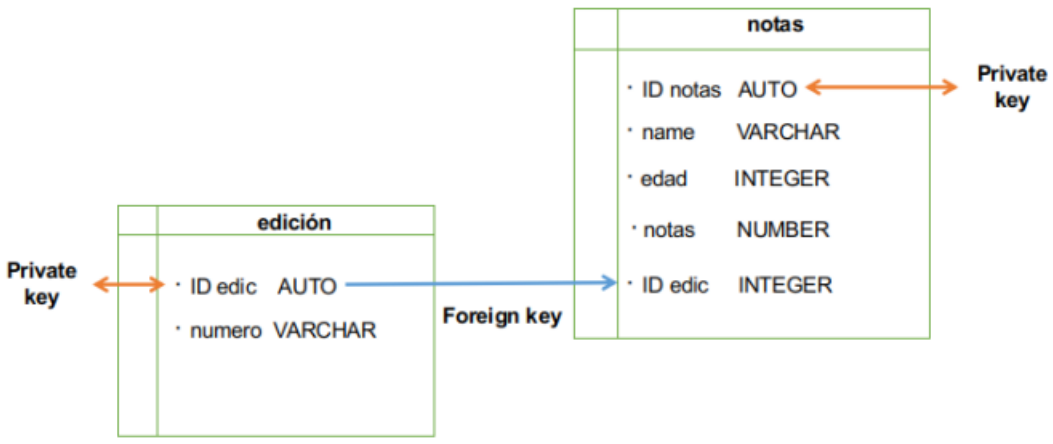
Nota: https://sqlite.org/lang_createtable.html

AYUDA Para la tabla edicción: "INSERT INTO edicion (numero) VALUES('{0}');".format(num)

Realizaremos con ayuda de python todo este proceso:

[1]:
```
from IPython.display import Image
Image('tabla_exp.png')
```

[1]:



Lo primero será crear la conexión a la base de datos:

[21]:
```
import sqlite3


conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

# Close communication with the database
conn.commit()
cur.close()
conn.close()
```

Insertar los siguientes valores de la tabla edición:

[2]:
```
Image("tabla_edic.png")
```

[2]:

edición	
ID edic	Número
1	Uno
2	Dos
3	Tres

Insertar los siguientes valores de la tabla notas:

[3]:
```
Image("tabla_notas.png")
```

[3]:

notas				
ID notas	**Name**	**Edad**	**Notas**	**ID edic**
1	Isabel Maniega	30	5.6	1
2	José manuel Peña	30	7.8	1
3	Pedro López	25	5.2	2
4	Julia García	22	7.3	1
5	Amparo Mayora	28	8.4	3
6	Juan Martinez	30	6.8	3
7	Fernando lópez	35	6.1	2
8	María Castro	41	5.9	3

Creamos un listado para los valores de la tabla para una mejor gestión:

[2]:
```
ediciones = ["UNO", "DOS", "TRES"]

notas = [{"name": 'Isabel Maniega', "age": 30, "notas": 5.6, "id_ed": 1},
         {"name": 'José Manuel Peña', "age": 30, "notas": 7.8, "id_ed": 1},
         {"name": 'Pedro López', "age": 25, "notas": 8.4, "id_ed": 2},
         {"name": 'Amparo Mayora', "age": 28, "notas": 7.3, "id_ed": 3},
         {"name": 'Juan Martínez', "age": 30, "notas": 6.8, "id_ed": 3},
         {"name": 'Fernando López', "age": 35, "notas": 6.1, "id_ed": 2},
         {"name": 'María Castro', "age": 41, "notas": 5.9, "id_ed": 3},
         ]
```

Crearemos primero la tabla edición, ya que la tabla Notas necesita de la tabla edición:

[4]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def createTablaNotas(cur):
    try:
        cur.execute("CREATE TABLE notas(ID_notas INTEGER PRIMARY KEY,\
                    name varchar(80), age int, grades real,\
                    ID_edic INT REFERENCES edicion(ID_edic));")
    except sqlite3.OperationalError as e:
        print("Error crear la tabla Notas: %s" % str(e))
```

```
def createTablaEdicion(cur):
    try:
        cur.execute("CREATE TABLE edicion(ID_edic INTEGER PRIMARY KEY,\
                    Numero varchar(80));")
    except sqlite3.OperationalError as e:
        print("Error crear la tabla Edicion: %s" % str(e))

# createTablaEdicion(cur)
# createTablaNotas(cur)

# Close communication with the database
cur.close()
conn.close()
```

Para el ID autoincrementado pondremos: INTEGER y el tipo de clave que es: PRIMARY KEY

En el caso de ID_edic podemos hacer referencia al ID_edic de la tabla edición

Insertamos los valores en las tablas creadas anteriormente:

[9]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def insertarEdicion(cur, value):
    try:
        cur.execute(f"INSERT INTO edicion (Numero) VALUES('{value}');")
    except sqlite3.OperationalError as e:
        print('Error Insertar dato: %s', str(e))

def insertarEdic(cur, ediciones):
    for edicion in ediciones:
        insertarEdicion(cur,edicion)

def insertarNotas(cur, name:str, age:int, notas:float, ID_edic:int):
    try:
        cur.execute(f"INSERT INTO notas (Name, age, grades, ID_edic)\
                     VALUES('{name}', {age}, {notas}, {ID_edic});")
    except sqlite3.OperationalError as e:
        print("Error crear registro: %s" % str(e))

def insertNotas(cur, notas):
    for nota in notas:
        insertarNotas(cur, nota['name'], nota['age'], nota['notas'],␣
 ↪nota['id_ed'])
```

```
# insertarEdic(cur, ediciones)
# insertNotas(cur, notas)

# Close communication with the database
conn.commit()
cur.close()
conn.close()
```

Otra forma usando executemany

[6]:
```
import sqlite3

conn = sqlite3.connect('actividad.db')
c = conn.cursor()

ediciones = [("UNO",), ("DOS",), ("TRES",)]

notas = [('Isabel Maniega', 30, 5.6, 1),
         ('José Manuel Peña', 30,  7.8, 1),
         ('Pedro López', 25, 8.4, 2),
         ('Amparo Mayora', 28, 7.3, 3),
         ('Juan Martínez', 30, 6.8, 3),
         ('Fernando López', 35, 6.1, 2),
         ('María Castro', 41, 5.9, 3),
         ]
c.executemany("INSERT INTO notas (Name, age, grades, ID_edic)\
               VALUES(?, ?, ?, ?);", notas)
c.executemany("INSERT INTO edicion (Numero) VALUES(?);", ediciones)
conn.commit()
conn.close()
```

Actualizar los datos de las notas de estos dos alumnos:

[4]:
```
Image("tabla_actualizar.png")
```

[4]:

Notas				
ID notas	**Name**	**Edad**	**Notas**	**ID edic**
3	Pedro López	25	**6.4**	2
8	María Castro	41	**5.2**	3

[10]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def actualizar(cur, tabla:str, nota:float, name:str):
    try:
        cur.execute(f"UPDATE {tabla} SET grades={nota} WHERE name='{name}';")
    except sqlite3.OperationalError as e:
        print("Error actualizar registro: %s" % str(e))

# actualizar(cur, 'notas', 6.4, 'Pedro López')
# actualizar(cur, 'notas', 5.2, 'María Castro')

# Close communication with the database
conn.commit()
cur.close()
conn.close()
```

1.0.1 Realizar:

- Debemos a continuación leer los dato.

[13]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def mostrar(cur, tabla:str):
    try:
        cur.execute(f"SELECT * FROM {tabla};")
        rows = cur.fetchall()
        for row in rows:
            print(row)
    except sqlite3.OperationalError as e:
        print("Error mostrar registros: %s" % str(e))

# mostrar(cur, "notas")
# mostrar(cur, "edicion")

cur.close()
conn.close()
```

- Buscaremos las notas que se encuentren entorno a 5-6.5

[15]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def filtrar(cur, tabla:str,column:str, nota1:float, nota2:float):
```

```
    cur.execute(f"SELECT * FROM {tabla} WHERE {column}\
                  BETWEEN {nota1} AND {nota2};")
    rows = cur.fetchall()
    print(f"\n---- Datos en tabla {tabla} con\
            {column} entre {nota1} y {nota2}--\n")
    for row in rows:
        print(row)

# filtrar(cur, "notas", "grades", 5.0, 6.5)

cur.close()
conn.close()
```

- Buscaremos los alumnos pertenecientes a la edicción DOS

Nota: https://www.w3schools.com/sql/sql_join.asp

[17]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def unirTablas(cur, tabla:str, value:str):
    cur.execute(f"SELECT * FROM {tabla} n INNER JOIN edicion e\
                  ON n.ID_edic = e.ID_edic WHERE numero = '{value}';")
    rows = cur.fetchall()
    print("\n---- Datos en de personas en la edicion DOS---\n")
    for row in rows:
        print(row)

# unirTablas(cur, "notas", 'DOS')

cur.close()
conn.close()
```

- Eliminar al alumno: “Pedro López”

[30]:
```
conn = sqlite3.connect("actividad.db")

cur = conn.cursor()

def delete_data(cur, tabla:str, name:str):
    try:
        cur.execute(f"DELETE FROM {tabla} WHERE name=?;", (name,))
    except sqlite3.OperationalError as e:
        print('Error eliminar dato: %s', str(e))

delete_data(cur, 'notas', "Pedro López")

conn.commit()
```

```
cur.close()
conn.close()
```

Creado por:

Isabel Maniega